

Qlassical: Diario di Bordo Frontend

3 aprile 2026

1 Data-Oriented Design (DOD) vs OOP

La scelta più radicale è stata rigettare la programmazione orientata agli oggetti (OOP) classica. Un approccio OOP avrebbe modellato le porte logiche come un'interfaccia base (**Gate**) con derivazioni polimorfiche (**HadamardGate**, **CnotGate**, ecc.), allocate dinamicamente sull'heap tramite `std::unique_ptr`.

Questo avrebbe causato *pointer chasing* e *cache miss* continui durante l'iterazione dell'Execution Engine. Abbiamo invece adottato il **Data-Oriented Design (DOD)**: il circuito è un singolo `std::vector` piatto e contiguo di strutture dati a dimensione fissa (**GateInstr**). L'iterazione da parte della CPU è rigorosamente lineare, massimizzando il prefetching della cache L1.

2 Struttura della Intermediate Representation (IR)

2.1 IR Hardcoded ed Estensibilità

Per evitare l'overhead del polimorfismo, l'IR è "hardcoded" tramite un'enumerazione fortemente tipizzata (**GateType**). Tuttavia, l'estensibilità del simulatore non è stata sacrificata: porte logiche arbitrarie (custom) o blocchi generati dal compilatore non richiedono nuove classi, ma vengono mappati sul tipo **UNITARY** o **FUSED_BLOCK**. La logica algebrica è de-accoppiata dall'istruzione e demandata a un *memory pool* separato. Inoltre è presente spazio libero nella codifica scelta che permette di aggiungere altre porte native con uno sforzo ragionevole.

2.2 Gestione a Livello di Byte (**GateInstr**)

L'istruzione centrale (**GateInstr**) è stata pensata per occupare esattamente **32 byte**, garantendo che due istruzioni fittino perfettamente in una singola cache line da 64 byte.

Il layout di memoria è il seguente:

- Byte 0 [**type**]: Enumeratore (1B) che identifica l'operazione.
- Byte 1 [**arity**]: Numero di qubit coinvolti (1B).
- Byte 2 [**flags**]: Bit-packed modifiers (es. Adjoint, Fused) (1B).
- Byte 3 [**_pad0**]: Padding per allineamento (1B).
- Byte 4--11 [**qubits**]: Array di 4 `int16_t` (8B). Espanso per supportare nativamente la Gate Fusion ottimale per la cache L1 ($k = 4$), evitando diramazioni condizionali costose (strategia Head/Tail) nel Backend.
- Byte 12--23 [**params**]: Array di 3 `float` per porte parametriche (12B).
- Byte 24--27 [**matrix.idx**]: Indice (4B) che punta al memory pool.
- Byte 28--31 [**uid**]: Identificatore univoco (4B) per la costruzione safe del DAG.

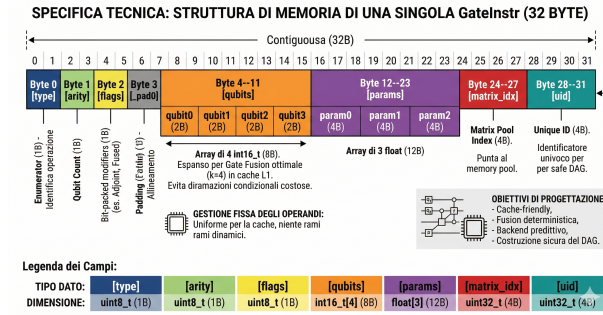


Figura 1: Risultato dopo litigi con Gemini per rappresentare una GateInstr

3 Memoria e Trasferimento Dati

3.1 Separazione Hot-Cold Data

Il simulatore separa rigorosamente i dati in base al loro pattern di accesso:

- **Hot Data (gate_stream):** Istruzioni iterate milioni di volte dal Middle-End e Backend. Mantengono leggere e inline.
- **Cold Data (unitary_pool):** Matrici complesse ($2^k \times 2^k$) necessarie solo durante l'applicazione di porte UNITARY. Conservate in un enorme array contiguo (`std::vector<std::complex<double>>`). Le `GateInstr` riferiscono queste matrici non tramite puntatori (che si invaliderebbero in caso di reallocazione), ma tramite offset logici (`matrix_idx`).

3.2 Zero-Copy e Move Semantics

La copia di strutture dati massicce (come l'intero circuito o il pool unitario) è bloccata a livello di compilazione (= `delete` su costruttori di copia). Per passare l'IR dal Frontend al Middle-End, o per trasferire le matrici ad engine di calcolo esterni (es. Eigen via `Eigen::Map`), si sfrutta la **Move Semantics** e le "viste" (`std::span`). Questo garantisce astrazioni *Zero-Copy*: i puntatori vengono trasferiti o affacciati sulla memoria esistente senza mai allocare nuovo spazio sull'heap.

4 Esecuzione: Algorithmic vs Data-Driven

Un'importante distinzione introdotta a livello architetturale è l'esecuzione ibrida nel Backend:

- **Algorithmic-Driven (Porte Standard):** Per gate come Hadamard o CNOT, l'IR non immagazzina alcuna matrice. Il Backend intercetta il tipo e lancia kernel intrinseci iper-ottimizzati, sostituendo l'uso passivo della banda di memoria (Memory Bandwidth) con calcolo puro (ALU).
- **Data-Driven (Custom/Fused):** Per blocchi matematici arbitrari generati dall'ottimizzatore, si usa un General Dense Matrix-Vector Kernel che attinge al Cold Data Pool.

5 Struttura Header-Only e Sistema di Build

Il modulo di Frontend è stato progettato come libreria **Header-Only**. Tale scelta favorisce l'inlining aggressivo da parte del compilatore C++20, eliminando l'overhead di chiamata a funzione durante la costruzione del circuito. L'intero progetto è gestito da CMake, garantendo build

riproducibili, flag di ottimizzazione rigorosi (`-O3, -march=native`) e integrazione automatizzata della suite di test (Catch2 v3).